

— Ui Dev Guide

Volume 1

30 + Angular Interview Questions

Ultimate Guide for 2024



What are the new features in angular 17 / which is the latest version of angular

- **New Syntax for Control Flow in Templates:** Angular v17 introduces a new declarative control flow syntax that makes writing and managing complex conditional codes easier.
- **Deferred loading:** This feature allows you to lazy load components and modules, which can improve the initial load time of your application.
- **Improved performance:** Angular 17 includes several performance improvements, including a new build system up to 87% faster than the previous build system.
- **Better support for server-side rendering (SSR):** Angular 17 includes several improvements to SSR, including support for deferred loading.
- **Standalone components:** Standalone components are a new type of component that can be used without a module. This can make it easier to create reusable components.
- **TypeScript 5.2 Support**

What are the different types of pipes?

Aspect	Pure Pipes	Impure Pipes
Change Detection	Pure pipes are called only when Angular detects a pure change	Impure pipes are called every time a change detection cycle is triggered
Performance	Typically more performant as they are called only when needed	Can potentially impact performance due to frequent calls
Caching	Angular caches the result of pure pipe transformation	Results are not cached, leading to repeated computation
Use Cases	Suitable for data transformation that doesn't change frequently	Ideal for operations involving side effects or asynchronous tasks

Pure Pipes:

- Called only when Angular detects a pure change in the input data.
- Typically more performant as they are called only when needed, reducing unnecessary computation.
- Angular caches the result of pure pipe transformations, improving efficiency by avoiding redundant calculations.
- Suitable for data transformation scenarios where the output is deterministic and doesn't change frequently.
- Best used for simple data manipulations or filtering operations on static data.

Impure Pipes:

- Called every time a change detection cycle is triggered, regardless of whether the input data has changed or not.
- May impact performance due to frequent execution, especially in scenarios with large datasets or complex computations.
- Results are not cached by Angular, leading to repeated computation for every change detection cycle.
- Ideal for operations involving side effects such as fetching data from external APIs, asynchronous tasks, or stateful transformations.
- Useful when the transformation logic depends on external factors that may change between change detection cycles.

How can we create a custom pipe in angular?

- **Create a Pipe Class:**
 - Implement PipeTransform interface from @angular/core.
 - Define a class with a transform method where the transformation logic is implemented.
- **Define Pipe Metadata:**
 - Use @Pipe decorator to define metadata for the custom pipe.
 - Specify the name property to reference the pipe in templates.
- **Implement the transform Method:**
 - Inside the transform method, define the logic to transform the input data.
 - The method takes the value to be transformed as the first argument and additional arguments if needed.
- **Register the Pipe:**
 - Register the custom pipe in an **Angular module** by adding it to the declarations array of the module if you are using module-based architecture.
 - if you have **standalone components** you have to add them to the imports array imports:[]

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-my-component',
  templateUrl: './my-component.component.html',
  styleUrls: ['./my-component.component.css'],
  imports: [PipeName] // need to import pipe at the top first
})
export class MyComponent {
  // Component logic goes here
}
```

- **Use the Pipe in Templates:**
 - Once registered, use the pipe name followed by the pipe operator (|) in templates.
 - Pass any necessary arguments to the pipe.

What is the use of a router outlet?

When you navigate through different views or pages within a **single-page application (SPA)**, the router outlet dynamically loads and renders the appropriate component based on the route specified in the URL. This allows for a seamless user experience where the entire page isn't reloaded every time the user navigates to a different section of the application.

For example, in Angular, you would use the `<router-outlet></router-outlet>` directive to define where the content of routed components will be inserted. Similarly, in Vue.js, you would use the `<router-view></router-view>` component for the same purpose.

Is it possible to use multiple router outlets in angular

No, it's not possible to use multiple router outlets directly in Angular. Router outlets serve as placeholders for rendering routed components within your application, and by design, Angular only supports a single primary router outlet per application.

However, you can achieve similar functionality by using named router outlets, which allow you to define multiple outlets within a single Angular component. This is useful when you want to render different components in different parts of your application layout.

Define named router outlets in your main template:

```
<router-outlet></router-outlet>
<router-outlet name="sidebar"></router-outlet>
```

In your routing configuration, specify the named outlets:

```
const routes: Routes = [
  { path: '', component: MainComponent },
  { path: 'sidebar', component: SidebarComponent, outlet: 'sidebar' },
  // Other routes...
];
```

Navigate to the named outlet in your component or programmatically:

```
<a [routerLink]="[{ outlets: { sidebar: 'sidebar' } }]">Show Sidebar</a>
```

This will render the SidebarComponent within the named router outlet named 'sidebar'.

How do you make routing using an anchor tag?



```
<a [routerLink]="[{ outlets: { sidebar: 'sidebar' } }]">Show Sidebar</a>
```



```
<a [routerLink]="['../parent']">Go to Parent</a>
```



```
<a [routerLink]="['/product', productId]">View Product</a>
```

Difference between `navigate()` and `navigateByUrl()`?

Feature	<code>navigate()</code>	<code>navigateByUrl()</code>
Usage	Navigates to a route based on route path segments or URL segments	Navigates to a route based on an absolute URL
Accepts	The array of route path segments or URL string	Absolute URL string
Route Path	Constructs the destination URL based on provided route path segments	N/A (Directly navigates to the provided absolute URL)
Navigation Extras	Optional navigation extras objects can be provided	Optional navigation extras objects can be provided
Example	<code>this.router.navigate(['/about']);</code>	<code>this.router.navigateByUrl('https://example.com');</code>

Using **`router.navigateByUrl`** is similar to changing the location bar directly—we are providing the “whole” new URL. Whereas **`router.navigate`** creates a new URL by applying an array of passed-in commands, a patch, to the current URL.

What is the difference between `@ViewChild` vs `@ViewChildren`?

In Angular, **`@ViewChild`** and **`@ViewChildren`** are decorators used to query and access elements or directives from the template in the component class. They serve similar purposes but have differences in how they handle querying for elements.

`@ViewChild`:

- @ViewChild allows you to obtain a reference to a single element or directive from the template.
- It queries for the first element or directive that matches the selector from the template.
- It returns the first instance of the queried element or directive or undefined if none is found.
- When using @ViewChild, you have to specify the type of the element or directive you are querying for.

```
import { ViewChild, ElementRef } from '@angular/core';

@Component({
  template: '<div #myElement>Some Element</div>',
  // Component configuration
})
export class MyComponent {
  @ViewChild('myElement') myElementRef: ElementRef;

  ngAfterViewInit() {
    // You can access the native element using this.myElementRef.nativeElement
  }
}
```

@ViewChildren:

- @ViewChildren allows you to obtain a reference to multiple elements or directives from the template.
- It queries for all elements or directives that match the selector from the template and returns a QueryList.
- It returns a QueryList even if there is only one element matching the selector.
- You can iterate through the QueryList to access each instance of the queried elements or directives.

```
import { ViewChildren, QueryList, ElementRef } from '@angular/core';

@Component({
  template: '<div #myElement>Element 1</div>\n<div #myElement>Element 2</div>',
  // Component configuration
})
export class MyComponent {
  @ViewChildren('myElement') myElementRefs: QueryList<ElementRef>;

  ngAfterViewInit() {
    // You can access each native element using this.myElementRefs.toArray()
  }
}
```

In summary, @ViewChild is used to query for a single element or directive, while @ViewChildren is used to query for multiple elements or directives. They both offer ways to interact with elements or directives defined in the component's template from the component class.

Difference between Observables and Promise

Feature	Promise	Observable
Execution	Eager	Lazy
Handling Multiple Values	Single value (fulfilled or rejected)	Multiple values over time
Cancellation	No support	Supports cancellation with <code>unsubscribe()</code>
Error Handling	<code>catch()</code> method	error callback in <code>subscribe()</code> method, <code>catchError()</code> operator
Chaining	<code>then()</code> method	Operators like <code>map()</code> , <code>filter()</code> , <code>mergeMap()</code> , etc.
Stream of Data	Represents a single value	Represents a stream of data over time
Caching	No support	Supports caching with operators like <code>shareReplay()</code>
Interoperability	Can be converted to an Observable using <code>from()</code>	Can be converted to a Promise using <code>toPromise()</code>

What is the use of a template in angular?

In Angular, templates are fundamental parts of components. They define the structure of what the user sees. Templates are written in HTML enhanced with Angular directives and binding syntax.

UI Rendering: Templates define the user interface (UI) of an Angular application. They specify how data and components should be presented to the user. HTML elements, attributes, and Angular directives are used to structure and style the UI.

Data Binding: Templates enable data binding, allowing the application to display dynamic data. Angular provides various types of data binding, including interpolation (`{{ }}`), property binding (`[ ]`), event binding (`(( ))`), and two-way binding (`[( )]`). These bindings connect the template with the component's data and enable synchronization between the view and the model.

Directives: Templates support Angular directives, which are special markers in the DOM that tell Angular's compiler to do something with a DOM element. Directives like `ngIf`, `ngFor`, and `ngSwitch` are commonly used in templates to conditionally render elements, loop over collections, and switch between multiple views.

Component Composition: Templates allow for the composition of components. Components can be nested within other components, creating complex UIs from smaller, reusable building blocks. Each component has its template, enabling encapsulation and separation of concerns.

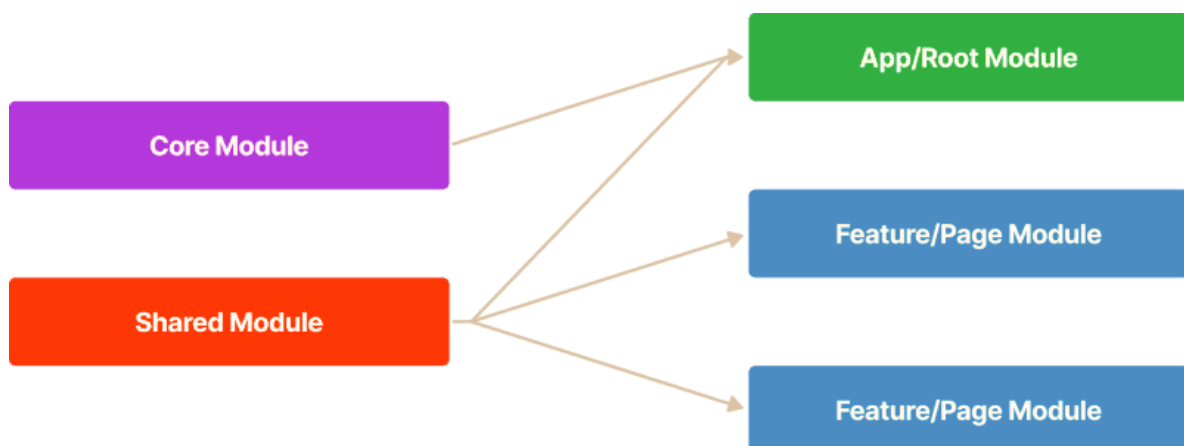
Event Handling: Templates handle user interactions through event binding. Event binding allows you to listen for user actions such as clicks, keystrokes, or mouse movements and respond to them by invoking methods defined in the component class.

Form Handling: Templates facilitate creating and validating forms in Angular applications. Angular provides directives and form controls that can be used in templates to create forms, handle user input, and perform validation.

What is the use of a shared module?

Shared Module will contain declarations for reusable components, directives, and pipes, as well as any common services that multiple feature modules may require. It may also export third-party modules, like Angular Material or ngx-translate, that are used throughout the application.

To use the Shared Module in other modules, it must be imported into each module that requires access to the shared components, directives, pipes, or services



The Benefits

It's usually a common requirement of many projects to have customized building blocks for the applications such as usual UI elements like buttons, dropdowns, alerts, cards, tables, checkboxes, text inputs, file inputs, select elements, and so on.

One of the key benefits of using a SharedModule is that it allows you to consolidate your presentational dummy components, which can be treated as a 3rd party library project. This makes it easier to manage and reuse your components across multiple projects (if needed), as well as share them with other developers or teams.

You can easily update and test your components in isolation, and make changes without affecting the functionality of your main application.

How do you create a material tab <mat-tab-group> <mat-tab>

First, install the angular material using the below-mentioned command.

ng add @angular/material

After completing the installation, Import 'MatTabsModule' from '@angular/material/tabs' in the app.module.ts file.

Then use the <mat-tab-group> tag to group all the items inside this group tag.

Inside the <mat-tab-group> tag we need to use <mat-tab> tag for every item.

We can also style the tabs based on the position like start, center, and end. For this, we need to use a property called mat-align-tabs.

If we want to change the theme then we can change it by using the color property. In angular we have 3 themes, they are primary, accent, and warn.

How do you create a dialog box MatDialogModule, MatDialog, MatDialogRef onclick we can open the dialog by passing the example

First, make sure you have imported MatDialogModule in your Angular module where you want to use dialogs. Then, you can inject MatDialog into your component where you want to open the dialog.

Importing Modules	Creating the Component for the Dialog Content:
<pre>import { NgModule } from '@angular/core'; import { BrowserModule } from '@angular/platform-browser'; import { MatDialogModule } from '@angular/material/dialog'; import { BrowserAnimationsModule } from '@angular/platform-browser/animations'; @NgModule({ imports: [BrowserModule, BrowserAnimationsModule, MatDialogModule], declarations: [AppComponent], bootstrap: [AppComponent] }) export class AppModule {}</pre>	<pre>import { Component } from '@angular/core'; @Component({ selector: 'app-dialog-content', template: `<h2 mat-dialog-title>Dialog Title</h2> <mat-dialog-content>   This is the dialog content. </mat-dialog-content> <mat-dialog-actions>   <button mat-button mat-dialog-close>Close</button> </mat-dialog-actions>` }) export class DialogContentComponent {}</pre>

Creating the Component to Open the Dialog

```

import { Component } from '@angular/core';
import { MatDialog } from '@angular/material/dialog';
import { DialogContentComponent } from './dialog-content.component';

@Component({
  selector: 'app-root',
  template: `<button (click)="openDialog()">Open Dialog</button>`
})
export class AppComponent {
  constructor(private dialog: MatDialog) {}

  openDialog(): void {
    const dialogRef = this.dialog.open(DialogContentComponent);

    dialogRef.afterClosed().subscribe(result => {
      console.log('Dialog closed:', result);
    });
  }
}

```

- We import the necessary modules (MatDialogModule, BrowserAnimationsModule) in the app module.
- We create a component DialogContentComponent for the content of the dialog.
- In AppComponent, we inject MatDialog and define a method openDialog() to open the dialog.
- When the button is clicked, openDialog() is called, which opens the dialog and subscribes to the afterClosed() event to handle the result when the dialog is closed.

Difference between ngAfterViewInitvs ngOnInit?

	ngOnInit	ngAfterViewInit
When Called	Once Angular has initialized all data-bound properties.	Once after Angular has initialized a component's view and its child views.
Purpose	Used for general initialization tasks.	Used for accessing the DOM or child components after they have been initialized.
Typical Use Cases	- Fetching initial data from a server. - Setting up subscriptions. - Performing other initialization tasks.	- Manipulating the DOM. - Interacting with child components. - Accessing the initialized view.
Access to DOM	No access to the DOM, as it's not yet initialized.	Access to the initialized DOM and child components.
ViewChild/ContentChild	ViewChild and ContentChild queries return undefined in ngOnInit.	ViewChild and ContentChild queries return initialized elements in ngAfterViewInit.

How do we invoke backend services using angular?

```

1. ng generate service data
2. Import HttpClientModule
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class DataService {

  constructor() { }

}

3. // In app.module.ts
import { HttpClientModule } from '@angular/common/http';

4. Inject HttpClient into Your Service
constructor(private http: HttpClient) { }

5. Create a Method for GET Requests

import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class DataService {
  constructor(private http: HttpClient) { }

  getData(): Observable<any> {
    return this.http.get('https://jsonplaceholder.typicode.com/todos/1');
  }
}

6. Call the API from a Component

export class AppComponent implements OnInit {
  data: any;

  constructor(private dataService: DataService) {}

  ngOnInit() {
    this.dataService.getData().subscribe(response => {
      this.data = response;
      console.log(this.data);
    });
  }
}

7. Display the Data in Your Component
<!-- In app.component.html -->
<div *ngIf="data">
  <h2>Data from API:</h2>

```

What is the purpose of the tsconfig.json file in Typescript?

The tsconfig.json file is a configuration file for TypeScript projects. It contains a set of options that control how the TypeScript compiler behaves when it transpiles TypeScript code to JavaScript.

The file is typically located in the root of a TypeScript project, and its name is always tsconfig.json.

Here is an example of what a tsconfig.json file might look like:



```

{
  "compilerOptions": {
    "target": "es5",
    "module": "commonjs",
    "outDir": "dist",
    "strict": true
  },
  "include": [
    "src/**/*.ts"
  ],
  "exclude": [
    "node_modules",
    "dist"
  ]
}

```

1. **Compiler Options:** You can specify compiler options such as target ECMAScript version (target), module system (module), output directory (outDir), source map generation (sourceMap), strict type checking (strict), and many others. These options control how TypeScript files are compiled into JavaScript.
2. **File Inclusion/Exclusion:** You can specify which files to include or exclude from compilation using the include and exclude properties. This allows you to control which files are part of your TypeScript project.
3. **Project References:** If you have a large TypeScript project that consists of multiple subprojects or dependencies, you can use the references property to specify project references. This allows you to set up dependencies between projects and enable incremental builds.
4. **Paths Mapping:** You can use the paths property to define path mappings for module resolution. This is useful for specifying custom module resolution logic or mapping module paths to physical file paths.
5. **Type Declarations:** You can specify type declaration files (*.d.ts) using the types or typeRoots properties. This allows you to include type definitions for third-party libraries or custom type declarations.
6. **Project Configuration:** tsconfig.json serves as the central configuration file for TypeScript projects. It helps maintain consistency across the project and enables you to share project settings with other developers.

How do we build our application specific to environment

1. When you build your Angular application, you can specify the target environment using the **--configuration** flag along with the build command.
2. For example:
 - **ng build --configuration=production** to build for the production environment.
 - **ng build --configuration=staging** to build for the staging environment.
3. Angular CLI will automatically use the environment-specific configuration file (**environment.prod.ts**, **environment.staging.ts**, etc.) corresponding to the specified configuration.

What are different CLI commands?

1. **ng new <project-name>**: Creates a new Angular project with the specified name.
2. **ng serve**: Builds and serves your Angular application locally. It watches for changes to your files and automatically reloads the browser.
3. **ng generate <schematic> <name>**: Generates new Angular artifacts such as components, services, modules, directives, pipes, etc. Schematics include component, directive, service,

module, pipe, class, guard, interface, enum, library, application, workspace, and more.

4. **ng build**: Builds your Angular application for production. By default, it creates a build in the dist/ directory.
5. **ng test**: Runs unit tests using Karma and Jasmine.
6. **ng e2e**: Runs end-to-end tests using Protractor.
7. **ng lint**: Lints the Angular project files using TSLint or ESLint.
8. **ng update**: Updates your Angular dependencies and performs automated migrations to newer versions of Angular.
9. **ng add <collection><name>**: Adds new capabilities to your Angular project by installing and configuring third-party libraries, tools, or packages. For example, ng add @angular/material.
10. **ng serve --configuration <config>**: Builds and serves your Angular application using a specific configuration. This is useful for serving the application with environment-specific configurations, such as ng serve --configuration=production.
11. **ng build --configuration <config>**: Builds your Angular application using a specific configuration. This is useful for building the application for different environments, such as ng build --configuration=production.
12. **ng config**: Sets or gets Angular configuration values in the project's angular.json file.

Can we use one module component to another module components?

1. Export the component in the module's exports array.
2. Import the module where the component is defined into the module where you want to use it.
3. Use the exported component in the template of components declared within the importing module.

```
@NgModule({
  declarations: [TaskCardComponent],
  imports: [MdCardModule],
  exports: [TaskCardComponent] // <== export the component you want to use in another
  module
})
export class TaskModule{}
```

What is the difference between package.json and package-lock.json?

1. package.json:

- package.json is a manifest file for Node.js projects. It contains metadata about the project, such as its name, version, description, author, and scripts.
- One of the key purposes of package.json is to specify project dependencies. It lists all the dependencies the project requires, their versions and other metadata.
- Developers typically manually manage package.json, adding or removing dependencies as needed. Dependencies can be specified with version ranges, allowing for flexibility in installed versions.
- package.json also allows defining scripts for common tasks like building, testing, and running the application.

2. package-lock.json:

- package-lock.json is a file generated by npm to lock down the versions of dependencies installed in the project.
- It provides a deterministic way to install dependencies by recording the exact version numbers of all installed packages and their transitive dependencies.
- The purpose of package-lock.json is to ensure that every developer working on the project installs the same versions of dependencies, avoiding issues with different environments or non-reproducible builds.
- package-lock.json is automatically generated or updated whenever you run npm install or npm update. It should be committed to version control along with package.json to ensure consistent dependency management across environments and team members.

What is the use FormControl

- Represents a single input field in a form.
- Provided by the @angular/forms module.
- Used to track the value and validation status of form controls.
- Creates instances to represent form inputs such as text inputs, checkboxes, radio buttons, etc.
- Binds to form elements in HTML templates using the FormControl directive.
- Accesses the current value using the value property.
- Subscribes to value changes using the value changes observable.
- Sets or resets the value programmatically using the setValue() and reset() methods.
- Supports validation by adding validators such as Validators.required and Validators.email.
- Plays a fundamental role in Angular's reactive forms API for building and managing forms.

Dependency injection

Dependency Injection (DI) in Angular is a design pattern used to manage the dependencies of components in a more modular and maintainable way. It allows components to define their dependencies straightforwardly, making the code easier to understand, test, and maintain.

- **Service Providers:** In Angular, dependencies are typically provided through services. Services are TypeScript classes that provide specific functionality throughout your application. They can be injected into components, directives, or other services.
- **Injectors:** Angular's dependency injection system uses injectors to instantiate and manage the components and services in your application. Angular's injector hierarchy allows for different levels of scope for dependency resolution.
- **Inject Decorator:** In Angular, you use the @Injectable decorator to make a class available to the dependency injection system. This decorator tells Angular that the class can be injected with its dependencies.
- **Constructor Injection:** Dependencies are typically injected into a component or service through its constructor. Angular's DI system automatically resolves and injects the required dependencies when an instance of the class is created.

Can we inject the service without a constructor in the component?

Import the inject() function: Start by importing the inject() function from the '@angular/core' package in your TypeScript file where you intend to perform the injection.

```
import { inject } from '@angular/core';
```

Inject the Service: To inject the service using inject(), you simply pass the type of the service as a parameter to the function. This retrieves the service instance associated with that type from the injector. For example:

```
export class MyComponent {  
  private service: inject(MyService);  
  constructor() {  
    console.log(this.service);  
  }  
}
```

Use Cases for Injecting Services Without Constructors:

Avoiding Circular Dependencies: Direct constructor injection can result in circular dependencies when two services have mutual dependencies. Injecting services without constructors can help circumvent this issue.

Deferring Injection: There might be situations where you need to delay the injection of a service until a certain point in a component's lifecycle. For example, you may want to inject the service based on the value of an input property, which can be more easily achieved without constructors.

Injecting Services into Non-Component Classes: While Angular components are the most common recipients of service injections, you can also inject services into other types of Angular classes, such as directives, pipes, or modules, using this technique.

What are the features of angular 16? Do you know how the application loads in angular 16

- Angular Signals
- Server-Side Rendering
- Experimental Jest Support
- esbuild-Based Build System
- Required Inputs
- Router Inputs
- Standalone Project Support

- ❑ • Auto-importing of components and pipes through the language service.
- ❑ • Support for TypeScript 5.0, ECMAScript decorators, service workers, and SCP through the CLI.
- ❑ • CSP support for online styles.
- ❑ • Self-closing tags.
- ❑ • Cease of support for ngcc and TypeScript 4.8.

NEW



What's New in Angular v17?

- TypeScript 5.2
- New Declarative Control Flow
- Deferrable Loading
- Support for View Transition API
- Improved Server-Side Rendering
- Router Refactoring
- Upgrade to Node js 18.13.0

In how many ways we can create a form in angular
which one is better

	Template-driven Forms	Reactive Forms (Model-driven Forms)
Approach	Template-driven	Reactive
Binding	Uses ngModel for two-way data binding	Manually bind using FormControl and FormGroup API
Form Creation	Implicit, based on HTML structure and ngModel	Explicit, programmatically created using FormBuilder
Form Controls	Limited control over individual form controls	Full control over form controls and their behavior
Validation	Template-driven validation using HTML attributes	Programmatically define validation rules in TypeScript
Dynamic Forms	Less suitable for dynamic forms	Well-suited for dynamic forms with changing structure
Code Readability	Simplicity and less code in the template	More code in the component class for form creation
Mutable/Immutable Data Model	Mutable data model (directly updates the model)	Immutable data model (updates are done through objects)
Testing	Easier to test in isolation with TestBed	Testing involves working with form controls and groups

If we have a form how will you set the value?

```
this.formGroup.setValue({
  firstName: 'John',
  lastName: 'Doe'
});
```

```
this.formGroup.patchValue({
  firstName: 'Jane'
});
```

When to Use Each

setValue(): Use setValue() when you want to set all the values of a form group or control at once, ensuring that all fields are specified and any missing fields may cause errors.

patchValue(): Use patchValue() when setting one or more values of a form group or control selectively, updating only the specified fields and leaving the others untouched.

In most cases, patchValue() is more flexible and commonly used because it allows you to work with partial-form data without having to specify all fields. However, both methods have their use cases depending on the specific requirements of your application.

What are directives in angular

Component Directives: These are the most common type of directives in Angular. Components are directives with a template. It changes the structure of the DOM

Examples of structural directives are **ngIf**, **ngFor**, **ngSwitch**

```
*ngIf – adds or removes element from DOM.
*ngFor – renders list of elements on every iteration.
```

All structural Directives are preceded by Asterix (*) symbol.

Structural Directives: These directives change the DOM layout by adding, removing, or manipulating elements. They are applied as attributes to elements in the template, but they affect the structure (adding or removing elements) rather than the appearance or behavior.

```
ngStyle – used to apply styles that will change the appearance.
ngClass – used to apply CSS classes to change the appearance.
```

Attribute Directives: These directives change the appearance or behavior of a DOM element, component, or another directive. They are applied as attributes to elements in the template.

```
import { Directive, ElementRef, HostListener } from '@angular/core';

@Directive({
  selector: '[highlight]',
})
export class HighlightDirective {
  constructor(private eleRef: ElementRef) {}

  @HostListener('mouseover') onMouseOver() {
    this.eleRef.nativeElement.style.color = 'red';
  }
  @HostListener('mouseleave') onMouseLeave() {
    this.eleRef.nativeElement.style.color = 'black';
  }
}
```

What are host binding and host listener?

@HostBinding and @HostListener are decorators in Angular that allow you to interact with the host element of a directive or a component.

@HostBinding

The @HostBinding decorator allows you to set properties on the host element of the directive or component. It binds a property of the directive or component class to a property of the host element.

```
import { Directive, HostBinding } from '@angular/core';

@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective {
  @HostBinding('style.backgroundColor') backgroundColor = 'yellow';
}
```

In this example, @HostBinding('style.backgroundColor') binds the backgroundColor property of the directive class to the backgroundColor CSS property of the host element. So, when this directive is applied to an element, it will set the background color of the host element to yellow.

@HostListener

The @HostListener decorator allows you to listen for events on the host element of the directive or component.

```
import { Directive, HostListener } from '@angular/core';

@Directive({
  selector: '[appCustomDirective]'
})
export class CustomDirective {
  @HostListener('click', ['$event']) onClick(event: MouseEvent) {
    console.log('Host element clicked', event);
  }
}
```

In this example, @HostListener('click') listens for the click event on the host element of the directive. When the host element is clicked, the onClick method is called, and you can perform actions accordingly.

Do you know the features of standalone components

Standalone components are designed to be reusable and self-contained, meaning they don't have strong dependencies on other components or services within the application. These components are often created to perform specific tasks or display specific types of content and can be easily integrated into various parts of the application.

Examples of standalone components in Angular could include a date picker, a modal dialog, a notification component, a progress bar, or a file uploader. These components can be developed independently and then integrated into different parts of the application as needed, providing modularity and maintainability to the Angular application.

Key characteristics of standalone components in Angular may include:

Isolation: Standalone components should be designed to function independently, which means they should not rely on the global state or other components to work properly.

Reusability: They are built with reusability in mind, so they can be used across different parts of the application or even in different Angular applications altogether.

Encapsulation: Standalone components encapsulate their functionality and may expose input and output properties to interact with the parent components or the application. This allows for customization and interaction while maintaining isolation.

Minimal Dependencies: These components should have minimal dependencies on external libraries or services, making them easy to integrate into different environments.

Clear Responsibilities: Standalone components often have a single responsibility or a well-defined set of responsibilities, which makes them easier to understand and maintain.

Independent Styling: They may also have their own CSS or styling encapsulated within the component to prevent styles from leaking into other parts of the application.

What are differ blocks?

the deferred loading blocks: Angular 17 is going to have a `@defer` control block enabling lazy-loading of the block's content. Lazy-loading also applies to the dependencies of the content of the block: all the components, directives and pipes are going to be lazy-loaded,

@Placeholder

```
@defer {  
  <large-component />  
} @placeholder (minimum 500ms) {  
  <p>Placeholder content</p>  
}
```

@loading

```
defer {  
  <large-component />  
} @loading (after 100ms; minimum 1s) {  
    
}
```

@viewport

```
@defer (on viewport) {  
  <calendar-cmp />  
} @placeholder {  
  <div>Calendar placeholder</div>  
}
```

@error

```
@defer {  
  <calendar-cmp />  
} @error {  
  <p>Failed to load the calendar</p>  
}
```

@Triggers

```
@defer (on viewport; on timer(5s)) {  
  <calendar-cmp />  
} @placeholder {  
    
}
```

many More do checkout

<https://angular.dev/guide/defer#loading>

What are signals?

The Angular16 is released with a lot of new features including Angular signals. This can be a game changer for Angular. Angular signals add reactivity to the Angular application

Angular depends on change detection and zone.js to detect changes and update the view.

The zone.js is a third-party library.

It works well with Angular applications but as Angular applications scale, it also becomes the bottleneck impacting the performance.

```
const count = signal(0);  
  
// Signals are getter functions - calling them reads  
// their value.  
console.log('The count is: ' + count());  
  
//setting values  
count.set(3);  
  
//updating value  
// Increment the count by 1.  
count.update(value => value + 1);
```

Writable signals provide an API for updating their values directly. You create writable signals by calling the signal function with the signal's initial value:

```
const count: WritableSignal<number> = signal(0);  
const doubleCount: Signal<number> = computed(() =>  
  count() * 2);
```

A computed signal derives its value from other signals. Define one using computed and specifying a derivation function:

The doubleCount signal depends on count. Whenever count updates, Angular knows that anything which depends on either count or doubleCount needs to update as well.

```
effect(() => {  
  console.log(`The current count is: ${count()}`);  
});
```

Signals are useful because they can notify interested consumers when they change. An effect is an operation that runs whenever one or more signal values change. You can create an effect with the effect function:

Effects always run at least once. When an effect runs, it tracks any signal value reads. Whenever any of these signal values change, the effect runs again.

What is interceptor?



10

11

12

an interceptor is a middleware-like service that intercepts HTTP requests and responses. Interceptors allow you to modify or handle HTTP requests and responses globally across your application. They provide a centralized location to implement common behavior such as adding headers, logging requests, handling errors, or transforming data.

13

14

15

16

17

18

Interceptors are implemented using Angular's `HttpInterceptor` interface, which defines two methods: `intercept` for intercepting HTTP requests and `interceptResponse` for intercepting HTTP responses.

How to add a header in the request



10

11

12

13

14

15

16

17

18

```
const headersConfig = {
  'Accept': 'application/json', //default headers
};
...

if (token) {
  headersConfig['Authorization'] = `Bearer ${token}`;
}
...

return next
  .handle(request.clone({
    setHeaders: headersConfig
  })))
```

Do you know rxjs operators?

Do you know what operator is mostly used from rxjs

RxJS (Reactive Extensions for JavaScript) is a library for reactive programming using Observables. RxJS provides a wide range of operators for creating, transforming, filtering, combining, and managing streams of data.

While the map operator is commonly used, there are many other operators in RxJS that serve different purposes, such as filter, mergeMap, switchMap, concatMap, combineLatest, zip, debounceTime, distinctUntilChanged, take, skip, and many more. The choice of which operator to use depends on the specific requirements and goals of your reactive programming application.

Do you know how to handle errors in services ?

handling errors in services typically involves using RxJS observables along with the catchError operator. Angular's HttpClient module returns observables for HTTP requests, making it convenient to handle errors using RxJS operators.

```
//imports...

export class MyService {
  constructor(private http: HttpClient) {}

  getData(): Observable<any> {
    return this.http.get<any>('https://api.example.com/data').pipe(
      catchError(this.handleError)
    );
  }

  private handleError(error: HttpResponse): Observable<any> {
    // Log the error to the console or handle it in any other way
    console.error('An error occurred:', error);

    // Return an observable with a user-facing error message
    return throwError('Something went wrong. Please try again later.');
```

- The **getData()** method sends an HTTP GET request using Angular's HttpClient and returns an observable. The **catchError** operator catches any errors that occur during the HTTP request.
- The **handleError()** method is a private method in the service that receives the error as an argument. You can customize this method to handle errors according to your application's needs.
- Inside **handleError()**, you can log the error or perform any other error-handling logic. Then, you return a new observable using **throwError** to propagate the error downstream.

Difference between pure pipes and impure pipes?

- Pure pipes are immutable and produce a new value when the input data changes.
- They are very efficient because they run only when their input arguments change.
- Pure pipes don't perform unnecessary computations if their input hasn't changed since the last change detection cycle.

```
12
13
14 import { Pipe, PipeTransform } from '@angular/core';
15
16 @Pipe({
17   name: 'evenNumbers'
18 })
19 export class EvenNumbersPipe implements PipeTransform {
20   transform(numbers: number[]): number[] {
21     return numbers.filter(num => num % 2 === 0)
22   }
23 }
24
25 <div *ngFor="let number of numbersArray | evenNumbers">
26   {{ number }}
27 </div>
```

- Impure pipes might be called more frequently as they are executed every time there is a change detection cycle, even if the input hasn't changed.
- They are useful for operations that have side effects, such as fetching data from a server or updating the DOM based on events.

```
import { Pipe, PipeTransform } from '@angular/core';
@Pipe({
  name: 'appendTimestamp',
  pure: false // This makes the pipe impure
})
export class AppendTimestampPipe implements PipeTransform {
  transform(value: string): string {
    return value + ' - ' + new Date().toISOString();
  }
}
<p>{{ message | appendTimestamp }}</p>
```

- Use pure pipes for operations that are deterministic and don't have side effects.
- Use impure pipes for operations that may change over time, involve asynchronous tasks, or have side effects.
- Be cautious when using impure pipes, as they can degrade performance if overused.

What are standalone components?

10
11
12
13
14
15
16
17
18

Do you know SSR

Do you know Ngrx



10

11

12

13

14

14

16

17

18

Server-Side Rendering (SSR) in Angular involves rendering Angular applications on the server-side before sending them to the client's browser. This approach improves performance, enhances SEO, and boosts accessibility. Angular Universal is the official solution for implementing SSR in Angular applications, providing tools and libraries to simplify the process. While SSR adds complexity and challenges, it offers significant benefits for modern web applications.

Ngrx provides a powerful and scalable solution for managing application state in Angular applications, making it easier to build complex and maintainable applications by promoting a predictable and reactive approach to state management. However, it may introduce additional complexity, especially in smaller applications, so it's essential to weigh the benefits against the overhead for your specific use case.

Could you please clarify Performance optimization techniques in angular

- **Lazy Loading:** Load modules and components only when needed.
- **Minification and Uglification:** Reduce file sizes by compressing code.
- **Bundle Optimization:** Eliminate dead code and split bundles.
- **Ahead-of-Time (AOT) Compilation:** Pre-compile templates for faster rendering.
- **Tree Shaking:** Remove unused code to reduce bundle size.
- **Code Splitting:** Load code asynchronously to improve loading times.
- **Optimized Images:** Compress images for faster loading.
- **Caching and CDN:** Cache static assets for quicker retrieval.
- **Server-Side Rendering (SSR):** Render pages on the server to improve initial load times.
- **Lazy Loading Images and Components:** Load content as needed for faster initial page load.
- **Optimized Data Binding:** Minimize data binding overhead.
- **Performance Monitoring:** Use tools to identify and address performance bottlenecks.

Angular Injectable

`@Injectable()` is a decorator used to annotate a class as available to be provided and injected as a dependency. When you apply this decorator to a class, Angular's dependency injection system can instantiate it and inject any required dependencies into its constructor.

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class MyService {
  constructor() { }

  // Methods and properties of the service
}
```

In this example:

- `@Injectable()` marks `MyService` as a service that Angular should manage.
- `providedIn: 'root'` specifies that Angular should provide this service at the root level, making it available throughout the application.
- The class `MyService` can have methods and properties that other components or services can use.

Angular's dependency injection system will take care of creating an instance of `MyService` when it's needed and providing it to any component or service that requests it.

Angular's dependency injection system will take care of creating an instance of `MyService` when it's needed and providing it to any component or service that requests it.

ngOnInit vs constructor

	Constructor	ngOnInit
Execution Time	Runs before ngOnInit	Runs after constructor but before rendering
Initialization	Use for basic initialization tasks	Use for complex initialization tasks
Dependency Injection	Ideal for injecting dependencies	Properties are not fully initialized
Angular Interaction	Avoid interaction with Angular framework here	Use for operations depending on Angular's framework
Data Fetching	Not suitable for fetching data or subscriptions	Suitable for fetching data, subscriptions, etc.
Component Readiness	Component not fully initialized yet	Component fully initialized
Input Properties	Properties not fully available	Properties fully available after initialization

Use of ngOnDestroy

Unsubscribing from Observables to prevent memory leaks.

```
ngOnDestroy(): void {  
  this.subscription.unsubscribe();  
}
```

Clearing timers or intervals to avoid unnecessary processing.

```
ngOnDestroy(): void {  
  clearInterval(this.timer);  
}
```

Cleaning up resources held by the component, such as closing connections or releasing memory.

```
ngOnDestroy(): void {  
  // Release the allocated resource  
  this.myService.releaseResource(this.resource)  
}
```

Async Pipe

The async pipe in Angular is a built-in pipe that facilitates the subscription to observables or promises directly from the template. It allows you to handle asynchronous data in a more concise and readable way without explicitly subscribing to observables in the component class.

Here's how the async pipe works:

- **Subscription Handling:** The async pipe subscribes to an observable or a promise provided to it as input in the template.
- **Automatic Unsubscription:** When the component is destroyed, Angular automatically unsubscribes from the observable or cancels the promise, preventing memory leaks.
- **Data Presentation:** Once the asynchronous operation completes, the async pipe automatically updates the view with the emitted values, handling the change detection process internally.

Key benefits of using the async pipe include:

- Simplified Template Code:
- Automatic Unsubscription:
- Improved Readability:

```
<div *ngIf="userData$ | async as user">  
  <h2>{{ user.name }}</h2>  
  <p>{{ user.email }}</p>  
</div>
```

In this example:

`userData$` is observable in the component class that emits user data. The async pipe subscribes to `userData$` and automatically updates the template with the emitted user data. The `as-user` syntax assigns the emitted value to a local variable named `user`, allowing you to reference it within the template.

Pure Pipe vs impure Pipe

- Pure pipes are immutable and produce a new value when the input data changes.
- They are very efficient because they run only when their input arguments change.
- Pure pipes don't perform unnecessary computations if their input hasn't changed since the last change detection cycle.

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'evenNumbers'
})
export class EvenNumbersPipe implements PipeTransform {
  transform(numbers: number[]): number[] {
    return numbers.filter(num => num % 2 === 0);
  }
}

<div *ngFor="let number of numbersArray | evenNumbers">
  {{ number }}
</div>
```

- Impure pipes might be called more frequently as they are executed every time there is a change detection cycle, even if the input hasn't changed.
- They are useful for operations that have side effects, such as fetching data from a server or updating the DOM based on events.

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'appendTimestamp',
  pure: false // This makes the pipe impure
})
export class AppendTimestampPipe implements PipeTransform {
  transform(value: string): string {
    return value + ' - ' + new Date().toISOString();
  }
}

<p>{{ message | appendTimestamp }}</p>
```

- Use pure pipes for operations that are deterministic and don't have side effects.
- Use impure pipes for operations that may change over time, involve asynchronous tasks, or have side effects.
- Be cautious when using impure pipes, as they can degrade performance if overused.

AOT compilation in angular

Ahead-of-Time (AOT) compilation in Angular is a process of converting Angular components and templates into highly optimized JavaScript code during the build phase, before the browser downloads and runs the code. This is in contrast to Just-in-Time (JIT) compilation, where compilation occurs in the browser at runtime.

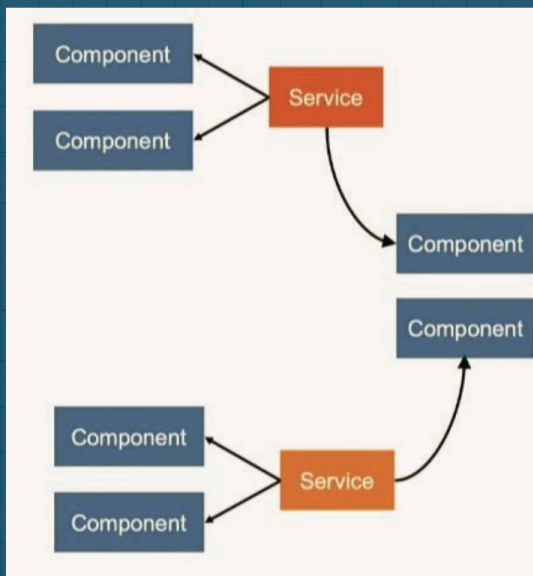
Performance: AOT compilation improves the performance of Angular applications by pre-compiling components and templates. This reduces the runtime overhead of parsing and compiling templates in the browser.

Bundle Size Reduction: AOT compilation removes Angular's template compiler from the final bundle, reducing the overall bundle size. This leads to faster download times and improved loading performance for users.

Error Detection: AOT compilation performs additional checks during the build process, such as template type checking and detecting errors in templates. This helps catch errors early in the development process, reducing the likelihood of runtime errors.

Security: AOT compilation eliminates the need to ship Angular's template compiler to the client, which improves the security of Angular applications by reducing the attack surface.

Data sharing in components



- Input and Output Properties (Parent-Child Communication)
- Service as a Data Store (Sibling Communication):
- RxJS Subjects or BehaviorSubjects (Cross-Component Communication)
- Redux or NgRx (Advanced State Management)
- LocalStorage or SessionStorage
- Router Parameters or Query Parameters
- ViewChild and ContentChild
- EventEmitter

Subject vs behavioral subject vs replay subject

Subject:

Basic implementation of the Subject class.

Does not retain the value emitted before the subscription.

Suitable for basic event emitting scenarios where the history of emitted values isn't required.

BehaviorSubject:

Extends the Subject and requires an initial value.

Subscribers receive the latest emitted value upon subscription.

Useful for managing application state, where subscribers need access to the current value.

ReplaySubject:

Also extends Subject and has a buffer to store emitted values.

Subscribers can receive a specific number of previous values emitted before subscription.

Useful for caching and replaying observable emissions, especially in scenarios like implementing a cache or logging functionality.

Server-Side Rendering (SSR)

Server-Side Rendering (SSR) in Angular involves rendering Angular applications on the server-side before sending them to the client's browser. This approach improves performance, enhances SEO, and boosts accessibility. Angular Universal is the official solution for implementing SSR in Angular applications, providing tools and libraries to simplify the process. While SSR adds complexity and challenges, it offers significant benefits for modern web applications.

Benefits of SSR:

- **Improved Performance:** SSR can improve the perceived performance of Angular applications by reducing the time to first render (TTFR). Users see content faster since the server sends rendered HTML immediately.
- **SEO:** Search engine crawlers can easily parse and index SSR-rendered pages, leading to better search engine rankings. SSR ensures that search engines see the fully rendered content without needing to execute JavaScript.
- **Accessibility:** SSR improves accessibility by ensuring that content is available to users who have JavaScript disabled or use assistive technologies.

Angular Universal:

- Angular Universal is the official solution for implementing SSR in Angular applications. It provides a set of tools and libraries that enable server-side rendering and prerendering of Angular applications.
- Angular Universal works seamlessly with Angular's existing features and ecosystem, allowing developers to build SSR-enabled Angular applications with minimal changes to their existing codebase.

Reactive form

VS

Template driven _form

Feature	Template-driven Forms	Reactive Forms (Model-driven Forms)
Approach	Template-driven	Reactive
Binding	Uses ngModel for two-way data binding	Manually bind using FormControl and FormGroup API
Form Creation	Implicit, based on HTML structure and ngModel	Explicit, programmatically created using FormBuilder
Form Controls	Limited control over individual form controls	Full control over form controls and their behavior
Validation	Template-driven validation using HTML attributes	Programmatically define validation rules in TypeScript
Dynamic Forms	Less suitable for dynamic forms	Well-suited for dynamic forms with changing structure
Code Readability	Simplicity and less code in the template	More code in the component class for form creation
Mutable/Immutable Data Model	Mutable data model (directly updates the model)	Immutable data model (updates are done through objects)
Testing	Easier to test in isolation with TestBed	Testing involves working with form controls and groups



ServerLess (Firebase)

Single-page applications (SPAs) written in JavaScript that provide the graphical user interface to your users,

Static file host to serve the JavaScript, CSS, and HTML files necessary for the SPA,

Cloud-based database that the SPA connects to directly, and finally,

functions-as-a-service provider that runs secure functions on a server.

ServerLess (Firebase)

Integrating Angular with Firebase, a serverless platform, allows you to build dynamic web applications without managing server infrastructure. Firebase offers various services such as hosting, real-time database, authentication, cloud functions, and more, which seamlessly integrate with Angular applications